Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Персистентность и изоляция: добавляем БД и настраиваем сети

Обучающийся: Тер-Степанян Эвелина Сергеевна

Номер ИСУ: 467697

Факультет: Факультет прикладной информатики

Группа: К3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Тер-Степанян Э. С.

дата, подпись

Руководитель _____ Аминов Н. С.

дата, подпись

Санкт-Петербург

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

ПЕРСОНАЛЬНЫЕ ПАРАМЕТРЫ **4**

1 Основная часть **5**

1.1 Этап 1. Запуск контейнера с именованным volume 5

1.2 Этап 2. Тест персистентности: пересоздание контейнера . . . 6

1.3 Этап 3. Bind mount для конфигурации 7

1.4 Этап 4. Пользовательские сети для изоляции 8

1.5 Этап 5. Финальный артефакт 10

ЗАКЛЮЧЕНИЕ **11**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **12**

ВВЕДЕНИЕ

Цель работы — подключить к приложению PostgreSQL, научиться хранить данные вне контейнера и разобраться с сетями Docker.

Задачи:

- создать именованный volume и запустить PostgreSQL с его использованием;
- подключить приложение к базе данных и проверить, что данные сохраняются после пересоздания контейнера;
- подключить файл конфигурации к контейнеру через bind mount;
- создать пользовательскую сеть и проверить сетевую изоляцию контейнеров.

Работа выполнялась на сервере linux-lab, Docker версии 29.1.3 [1].

ПЕРСОНАЛЬНЫЕ ПАРАМЕТРЫ

Персональные параметры приведены в таблице 1.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	467697
Персональный порт (assistant)	8097
Персональный порт (postgres)	5527
Персональный префикс	ter-stepanyan
Секретный ключ	secret97ter-stepanyan
Имя базы данных	ter-stepanyan97

Репозиторий с результатом (ветка `lab-3`): https://github.com/LilllDrake/KUBER/blob/lab-3/lab-2/assist-py/.

1 Основная часть

1.1 Этап 1. Запуск контейнера с именованным volume

Создаю именованный volume для БД и проверяю его создание на рисунке 1.

```
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker volume create ter-stepanyan-pgdata
 ter-stepanyan-pgdata
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker volume ls | grep ter-stepanyan-pgdata
 local     ter-stepanyan-pgdata
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker volume inspect ter-stepanyan-pgdata
 [
     {
         "CreatedAt": "2026-10-20T16:22:45Z",
         "Driver": "local",
         "Labels": null,
         "Mountpoint": "/var/lib/docker/volumes/ter-stepanyan-pgdata/_data",
         "Name": "ter-stepanyan-pgdata",
         "Options": null,
         "Scope": "local"
     }
 ]
 lildrake@linux-lab:~/KUBER/lab-2/assist-py$
```

Рисунок 1 — Создание и проверка тома

Запуск PostgreSQL с моим volume и проверка статуса представлены на рисунке 2.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
--name ter-stepanyan-postgres \
-e POSTGRES_USER=user_97 \
-e POSTGRES_PASSWORD=pgpass_97 \
-e POSTGRES_DB=assistant_97 \
-v ter-stepanyan-pgdata:/var/lib/postgresql/data \
-p 5527:5432 \
postgres:15
ebb8728362bb09b0bb9bcf32046a8c291e39ce3d85c777108faf11fc77386b2f
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker ps --filter "name=ter-stepanyan-postgres" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                  STATUS          PORTS
ter-stepanyan-postgres   Up 21 seconds   0.0.0.0:5527->5432/tcp, [::]:5527->5432/tcp
```

Рисунок 2 — Запуск PostgreSQL

Исходное приложение хранило заметки в списке в памяти процесса и не использовало переменную `DATABASE_URL`, поэтому после удаления контейнера заметки пропадали. В `app/main.py` обработчики `/notes` изменены так, чтобы заметки записывались в таблицу PostgreSQL, а в `requirements.txt` добавлен драйвер `psycopg`. Собираю образ приложения из `Dockerfile.optimized` на рисунке 3.

```
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker images | grep ter-stepanyan-assistant:v3
 WARNING: This output is designed for human readability. For machine-readable output, please use --format.
 ter-stepanyan-assistant:v3         94e640ebd31b        321MB         77.1MB
```

Рисунок 3 — Сборка образа

Запускаю ассистента и подключаю его к БД на рисунке 4. Строка подключения передаётся через переменную `DATABASE_URL`.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
  --name ter-stepanyan-assistant \
  -p 8097:8097 \
  -e APP_OWNER=ter-stepanyan \
  -e APP_SECRET=secret97ter-stepanyan \
  -e DATABASE_URL="postgresql://user_467697:pgpass_467697@host.docker.internal:5527/assistant_467697" \
  ter-stepanyan-assistant:v3
719386c6ab5bf6cebbb1d17bb45438a2ea546a498f74583028d0c4f186f6092e
```

Рисунок 4 — Запуск ассистента и подключение к БД

Проверяю работоспособность ассистента на рисунке 5: в ответе `/notes` видна созданная заметка.

```
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl http://localhost:8097
{"message":"👋 Привет, ter-stepanyan!","service":"personal-assistant","version":"1.0","endpoints":["/me","/notes","/weather/{city}","/secret/{key}"]}
/KUBER/lab-2/assist-py$
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl http://localhost:8097/health
{"status":"healthy","owner":"ter-stepanyan"}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl -X POST http://localhost:8097/notes \
  -H "Content-Type: application/json" \
  -d '{"text": "ЛР3: данные в БД"}'
{"status":"saved","note_id":0,"total":1}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl http://localhost:8097/notes
{"notes":["ЛР3: данные в БД"],"count":1}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
lildrake@linux-lab:~/KUBER/lab-2/assist-py$
```

Рисунок 5 — Проверка работоспособности

1.2 Этап 2. Тест персистентности: пересоздание контейнера

Удаляю контейнер с ассистентом и пересоздаю его на рисунке 6.

```
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker stop ter-stepanyan-assistant
ter-stepanyan-assistant
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker rm ter-stepanyan-assistant
ter-stepanyan-assistant
[lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
2ddd6180dd38   postgres:15   "docker-entrypoint.s…"   19 minutes ago   Up 19 minutes   0.0.0.0:5527->5432/tcp, [::]:5527->5432/tcp   ter-stepanyan-postgres
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
--name ter-stepanyan-assistant \
-p 8097:8097 \
-e APP_OWNER=ter-stepanyan \
-e APP_SECRET=secret97ter-stepanyan \
--add-host=host.docker.internal:host-gateway \
-e DATABASE_URL="postgresql://user_97:pgpass_97@host.docker.internal:5527/assistant_97" \
[ter-stepanyan-assistant:v3
b9eaf3f1257031c09112136504702860 73fa3742f873e95306b11cbf46ebacbb
```

Рисунок 6 — Пересоздание ассистента

Проверяю, что данные сохранились, и создаю артефакт на рисунке 7. Заметка осталась, потому что она хранится в PostgreSQL, а данные базы лежат в volume `ter-stepanyan-pgdata`, который не зависит от контейнера приложения.

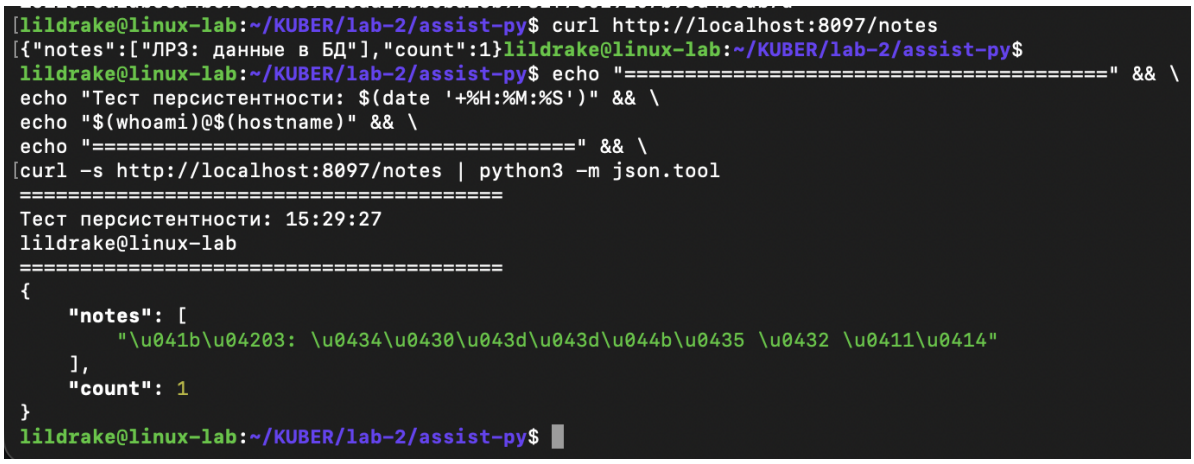


Рисунок 7 — Артефакт персистентности

1.3 Этап 3. Bind mount для конфигурации

Создаю папку `config`, в ней файл `app.conf` на рисунке 8.

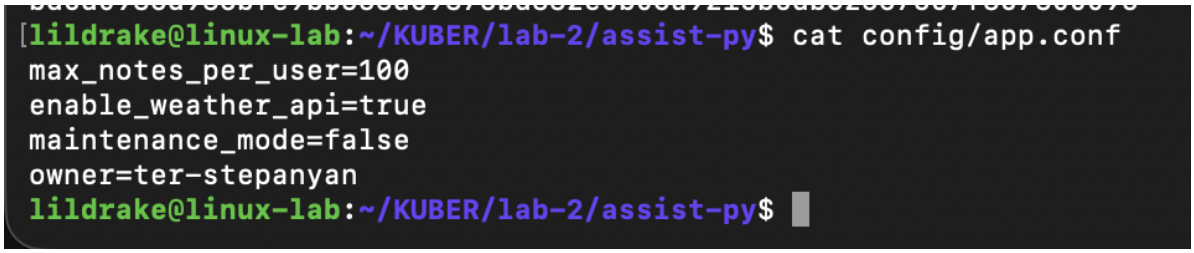


Рисунок 8 — Содержимое файла app.conf

Запускаю контейнер с bind mount на рисунке 9: локальная папка `config` монтируется внутрь контейнера по пути `/app/config` в режиме только для чтения. Также проверяю конфигурацию внутри контейнера.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
--name ter-stepanyan-assistant-config \
-p 8097:8097 \
-e APP_OWNER=ter-stepanyan \
-e APP_SECRET=secret97ter-stepanyan \
--add-host=host.docker.internal:host-gateway \
-e DATABASE_URL="postgresql://user_97:pgpass_97@host.docker.internal:5527/assistant_97" \
-v $(pwd)/config:/app/config:ro \
ter-stepanyan-assistant:v3
bdcd0936d936bfe9bb553dc9375bd332e0b06a921cbcdb625675c7f3c7300095
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ cat config/app.conf
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=false
owner=ter-stepanyan
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker exec -it ter-stepanyan-assistant-config sh
# cat /app/config/app.conf
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=false
owner=ter-stepanyan
# exit
lildrake@linux-lab:~/KUBER/lab-2/assist-py$
```

Рисунок 9 — Запуск контейнера с bind mount

На хосте в файле `config/app.conf` значение `maintenance_mode` изменено на `true`, после чего контейнер перезапущен командой `docker restart`. Проверка через `docker exec` на рисунке 10 показала, что внутри контейнера файл содержит новое значение без пересборки образа.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ nano config/app.conf
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ cat config/app.conf
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=true
owner=ter-stepanyan
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker restart ter-stepanyan-assistant-config
ter-stepanyan-assistant-config
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker exec ter-stepanyan-assistant-config cat /app/config/app.conf
max_notes_per_user=100
enable_weather_api=true
maintenance_mode=true
owner=ter-stepanyan
lildrake@linux-lab:~/KUBER/lab-2/assist-py$
```

Рисунок 10 — Проверка изменения значения

1.4 Этап 4. Пользовательские сети для изоляции

Создаю пользовательскую сеть и удаляю старые контейнеры на рисунке 11.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker network create ter-stepanyan-backend
ccbeb99f7165c6a16d41d04bb7ba25dd4d0a1b059b8f822ae8eafde10ca22f56
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker stop ter-stepanyan-postgres ter-stepanyan-assistant-config
ter-stepanyan-postgres
ter-stepanyan-assistant-config
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker rm ter-stepanyan-postgres ter-stepanyan-assistant-config
ter-stepanyan-postgres
ter-stepanyan-assistant-config
```

Рисунок 11 — Создание пользовательской сети

На рисунке 12 запускаю БД и ассистента в одной сети `ter-stepanyan-backend`. Ассистент подключается к базе по имени контейнера `ter-stepanyan-postgres` и порту 5432. База доступна только внутри сети, снаружи к ней подключиться нельзя.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
--name ter-stepanyan-postgres \
--network ter-stepanyan-backend \
-e POSTGRES_USER=user_97 \
-e POSTGRES_PASSWORD=pgpass_97 \
-e POSTGRES_DB=assistant_97 \
-v ter-stepanyan-pgdata:/var/lib/postgresql/data \
postgres:15
1d4c37bd754c66b15bc99311f587821424d032b307c92620e3569081acc8c544
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -d \
--name ter-stepanyan-assistant \
--network ter-stepanyan-backend \
-p 8097:8097 \
-e APP_OWNER=ter-stepanyan \
-e APP_SECRET=secret97ter-stepanyan \
-e DATABASE_URL="postgresql://user_97:pgpass_97@ter-stepanyan-postgres:5432/assistant_97" \
-v $(pwd)/config:/app/config:ro \
ter-stepanyan-assistant:v3
a12f13e097d4984f3d1d6fe1309b905df57a29f083e81d72413088c8ebfa7988
```

Рисунок 12 — Создание контейнеров для БД и ассистента

Проверяю работоспособность, создаю новую заметку и получаю все заметки на рисунке 13. Заметка с первого этапа тоже на месте.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl http://localhost:8097/health
{"status":"healthy","owner":"ter-stepanyan"}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl -X POST http://localhost:8097/notes -H "Content-Type: application/json" -d '{"text": "ЛР3: сеть работает!"}'
{"status":"saved","note_id":1,"total":2}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ curl http://localhost:8097/notes
{"notes":["ЛР3: данные в БД","ЛР3: сеть работает!"],"count":2}lildrake@linux-lab:~/KUBER/lab-2/assist-py$
```

Рисунок 13 — Проверка связности

Для исследования сетевой изоляции запускаю временные контейнеры на рисунке 14: первый вне сети, второй внутри сети. Контейнер вне сети не находит `ter-stepanyan-postgres` по имени (ошибка `bad address`), а контейнер внутри сети получает ответ на ping.

```
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -it --rm alpine:3.18 ping -c 2 ter-stepanyan-postgres
Unable to find image 'alpine:3.18' locally
3.18: Pulling from library/alpine
95459497489f: Pull complete
e03f60a96940: Download complete
f6a5f6578c75: Download complete
Digest: sha256:de0eb0b3f2a47ba1eb89389859a9bd88b28e82f5826b6969ad604979713c2d4f
Status: Downloaded newer image for alpine:3.18
ping: bad address 'ter-stepanyan-postgres'
lildrake@linux-lab:~/KUBER/lab-2/assist-py$ docker run -it --rm --network ter-stepanyan-backend alpine:3.18 ping -c 2 ter-stepanyan-postgres
PING ter-stepanyan-postgres (172.18.0.2): 56 data bytes
64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.074 ms
64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.158 ms

--- ter-stepanyan-postgres ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 0.074/0.116/0.158 ms
```

Рисунок 14 — Исследование сетевой изоляции

1.5 Этап 5. Финальный артефакт

На рисунке 15 представлена итоговая инфраструктура: контейнеры `ter-stepanyan-postgres` и `ter-stepanyan-assistant`, volume `ter-stepanyan-pgdata` и сеть `ter-stepanyan-backend`.

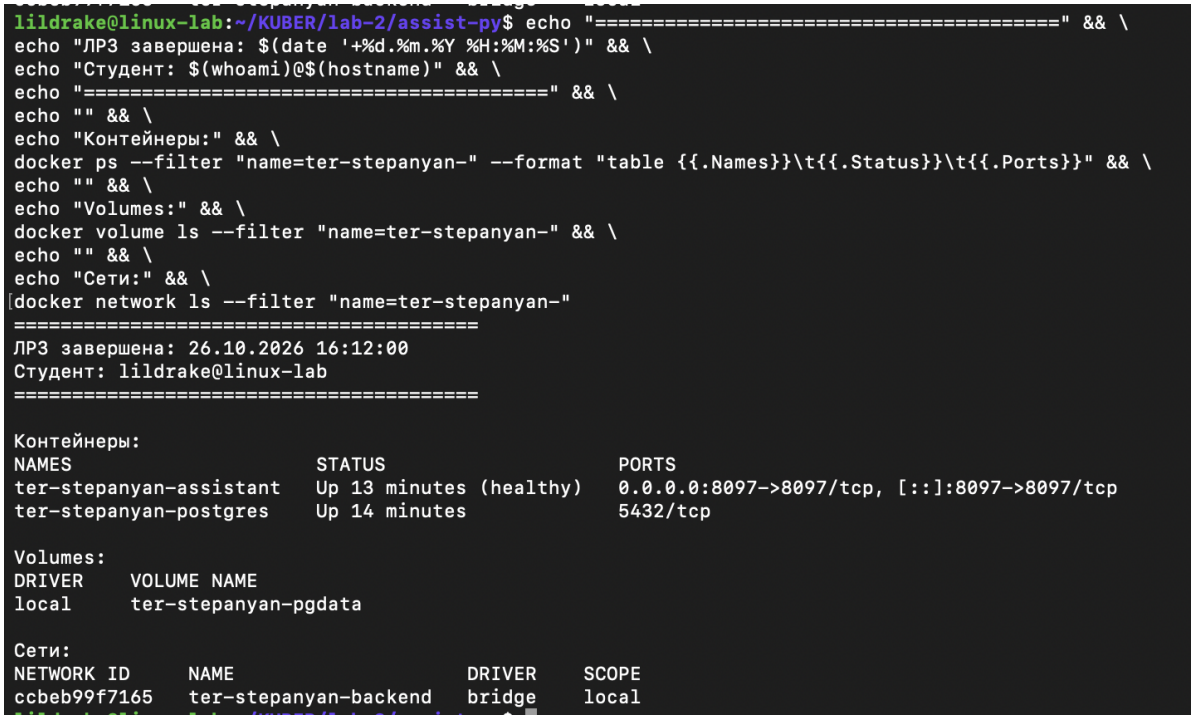


Рисунок 15 — Финальный артефакт

ЗАКЛЮЧЕНИЕ

В ходе работы приложение было подключено к PostgreSQL, а данные базы вынесены в именованный volume. Проверено, что заметки сохраняются после удаления и пересоздания контейнеров. С помощью bind mount файл конфигурации подключён к контейнеру без пересборки образа. Пользовательская сеть позволила контейнерам обращаться друг к другу по имени и изолировала их от контейнеров вне сети.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation: документация Docker. URL: https://docs.docker.com/ (дата обращения: 26.10.2026).